PREVIEW GERADO AUTOMATICAMENTE a partir de Relatorio_PAP_CyberSafe_LIVE.docx. Nao e a fonte do relatorio e nao substitui o Word. Baseline DOCX/PDF permanecem intocados.

Curso Profissional deProgramador/a de Informática

Prova de Aptidão ProfissionalAno letivo: 2024/2027

RELATÓRIO

CYBERSAFE ASSISTANT

Centro Inteligente de Segurança e Saúde do PC

AlunoCaio Cesar Silva Amorim

Trabalho realizado sob a orientação deProfessor André Meira

Versão revista e ampliada — 4 de outubro de 2026

DEDICATÓRIA

Dedico este projeto à minha família, à minha namorada e a todas as pessoas que me apoiaram ao longo do curso e do desenvolvimento da PAP. O apoio, a paciência e o incentivo recebidos foram essenciais para transformar uma ideia inicial num projeto que exigiu aprendizagem, persistência, capacidade de resolver problemas e evolução contínua.

AGRADECIMENTOS

Quero agradecer a todos os professores que contribuíram para o meu percurso e para o desenvolvimento deste projeto, através das suas orientações, sugestões, ideias e feedback.

Agradeço, em especial, ao Professor André Meira, enquanto professor orientador, pelo acompanhamento, pelas recomendações e pelo incentivo para analisar o projeto de forma crítica e explorar novas possibilidades de evolução.

Agradeço aos meus pais pelo apoio ao longo do curso e durante o desenvolvimento da Prova de Aptidão Profissional. Agradeço também à minha namorada pelo apoio constante e pela inspiração pessoal que esteve na origem da mascote Matrix.

Por fim, agradeço aos meus amigos que testaram o programa, partilharam opiniões e sugestões e ajudaram a identificar problemas. O contributo destas pessoas foi importante para melhorar o projeto, embora a conceção, o desenvolvimento, a integração e a validação final da aplicação tenham sido realizados individualmente por mim.

NOTA DE REVISÃO E ÂMBITO DESTA VERSÃO

Esta edição foi revista documentalmente em 4 de outubro de 2026. A revisão amplia o enquadramento teórico, acrescenta literatura científica recente, clarifica as sobreposições entre segurança, saúde, desempenho, privacidade e assistência, introduz princípios de desenvolvimento seguro, contexto regulamentar europeu, uma matriz de riscos e uma discussão mais estruturada dos resultados.

A atualização documental não altera retroativamente os factos técnicos que não tenham nova evidência. Os estados dos motores, contagens de testes, limitações do scanner e restantes resultados continuam ancorados no último checkpoint comprovado do projeto, sobretudo entre 30 de setembro e 1 de outubro de 2026. Sempre que uma capacidade permanece parcial, experimental, em validação ou futura, essa condição é mantida explicitamente.

> **Princípio de honestidade técnica —o relatório distingue aquilo que está implementado, aquilo que foi validado em runtime, aquilo que existe apenas como protótipo e aquilo que permanece planeado. "Sem deteções" não é tratado como sinónimo de "computador seguro".**

RESUMO

O presente relatório apresenta o desenvolvimento do CyberSafe Assistant, uma aplicação desktop modular para Windows concebida como Centro Inteligente de Segurança e Saúde do PC. O projeto evoluiu de uma proposta inicial de antivírus para uma plataforma que combina análise de ficheiros, estado dos mecanismos de proteção, monitorização do sistema, telemetria, processos, rede, quarentena, findings priorizados e assistência contextual. A arquitetura assenta em React, Electron, TypeScript, Node.js, comunicação IPC e named pipe, integrações com o Windows e um motor nativo complementar em C++.

No checkpoint técnico documentado, o ProtectionHealth distingue motores ativos, degradados, indisponíveis e em erro; o YARA-X encontra-se funcional mas com cobertura limitada, a base local de hashes contém quatro entradas ainda em revisão de proveniência, o Native Engine está operacional em desenvolvimento e o ClamAV permanece degradado por ausência da base de assinaturas. Foi corrigido um erro de SHA-256 no motor nativo, auditada a inexistência de contaminação dos dados persistidos e corrigido um falso positivo grave que classificava o executável legítimo Bun como ameaça crítica. O Gestor de Tarefas consegue terminar processos normais por PID com proteções contra processos internos ou críticos, embora a latência de atualização da lista permaneça instável.

A versão revista aprofunda ainda a fundamentação com o NIST Cybersecurity Framework 2.0, a revisão 3 da NIST SP 800-61, o Secure Software Development Framework, o AI Risk Management Framework, literatura científica sobre deteção de malware e usable security, o Regulamento Geral sobre a Proteção de Dados e o Regulamento de Ciber-Resiliência. Estas referências são utilizadas como enquadramento e não como declaração de conformidade. O projeto é avaliado segundo uma

lógica de evidência: cobertura, estado do motor, confiança, impacto para o utilizador e limitação conhecida são tratados como dimensões diferentes.

Palavras-chave: Cibersegurança; Segurança de endpoints; Monitorização; Windows; Inteligência artificial; Usable security; Aplicação desktop.

ABSTRACT

This report presents the development of CyberSafe Assistant, a modular Windows desktop application conceived as an Intelligent PC Security and Health Centre. The project evolved from an initial antivirus concept into a platform that combines file analysis, protection-engine health, system monitoring, telemetry, process and network information, quarantine, prioritised findings and contextual assistance. Its architecture uses React, Electron, TypeScript, Node.js, IPC and named-pipe communication, Windows integrations and a complementary native C++ engine.

At the documented technical checkpoint, ProtectionHealth distinguishes active, degraded, unavailable and error states; YARA-X is operational with limited rule coverage, the local hash database contains four entries under provenance review, the Native Engine is operational in development, and ClamAV remains degraded because its signature database is unavailable. Development also corrected a native SHA-256 defect, verified that the faulty digest had not contaminated persisted decisions, and fixed a serious false positive affecting the legitimate Bun executable. The Task Manager can terminate ordinary processes by PID with structured feedback and safety guards, while list-refresh latency remains unstable.

This revised edition expands the theoretical framework with NIST CSF 2.0, NIST SP 800-61 Revision 3, the Secure Software Development Framework, the AI Risk Management Framework, scientific literature on malware detection and usable security, the GDPR and the EU Cyber Resilience Act. These sources are used as conceptual references rather than claims of regulatory compliance. The report deliberately separates implementation, runtime validation, coverage, limitations and future work in order to avoid presenting unverified capabilities as effective protection.

Keywords: Cybersecurity; Endpoint security; Monitoring; Windows; Artificial intelligence; Usable security; Desktop application.

ÍNDICE

1. INTRODUÇÃO 1

1.1 Contexto, problema e proposta de valor 1

1.2 Âmbito, limites e questão orientadora 1

1.3 Organização do relatório 1

2. ENQUADRAMENTO TEÓRICO 2

2.1 Cibersegurança e gestão de risco 2

2.2 Malware e mecanismos de deteção 2

2.3 Proteção em camadas, cobertura e confiança 3

2.4 Monitorização de endpoint, telemetria e resposta a incidentes 3

2.5 Segurança utilizável e o utilizador no circuito 4

2.6 Inteligência artificial aplicada à assistência 4

2.7 Segurança em extensões de navegador 4

2.8 Privacidade, minimização e proteção de dados desde a conceção 5

2.9 Windows Firewall e Windows Filtering Platform 5

2.10 Segurança de aplicações Electron e comunicação IPC 5

2.11 Logging, rastreabilidade e qualidade da evidência 6

2.12 Desenvolvimento seguro e cadeia de fornecimento de software 6

2.13 Regulamento de Ciber-Resiliência: contexto europeu 7

2.14 Sobreposições entre segurança, saúde, desempenho, privacidade e assistência 7

2.15 Síntese teórica e artigos relacionados 9

3. DESENVOLVIMENTO DO PROJETO 11

3.1 Apresentação e evolução da ideia 11

3.2 Motivação e fundamentação 11

3.3 Objetivos 12

3.4 Público-alvo 12

3.5 Planeamento e metodologia 13

3.6 Ferramentas e tecnologias utilizadas 14

3.7 Arquitetura do sistema 15

3.8 Desenvolvimento da interface 16

3.9 ProtectionHealth e PcState17

3.10 Monitorização e PC Center18

3.11 Scanner e quarentena18

3.12 Matrix19

3.13 Módulos complementares19

3.14 Testes e validação20

3.15 Dificuldades encontradas e soluções aplicadas23

3.16 Avaliação provisória dos resultados25

3.17 Limitações atuais25

3.18 Registo de riscos técnicos26

3.19 Trabalho futuro27

4. AVALIAÇÃO E DISCUSSÃO28

4.1 Cumprimento dos objetivos28

4.2 Cobertura, confiança e honestidade técnica28

4.3 Usabilidade e comunicação de risco29

4.4 Segurança do próprio software e sustentabilidade29

4.5 Aprendizagens e valor da PAP29

5. CONCLUSÃO30

6. REFERÊNCIAS BIBLIOGRÁFICAS31

ANEXOS33

ÍNDICE DE QUADROS, TABELAS E FIGURAS

Quadro 1 — Mapeamento conceptual entre NIST CSF 2.0 e CyberSafe

Quadro 2 — Literatura e artigos técnicos utilizados no enquadramento

Quadro 3 — Sobreposições funcionais entre dados, interpretação, risco e orientação

Quadro 4 — Desenvolvimento seguro, privacidade e rastreabilidade aplicados ao projeto

Figura 1 — Ciclo conceptual do CyberSafe Assistant

Figura 2 — Arquitetura lógica e fronteiras de confiança

Figura 3 — Mapa conceptual de sobreposições funcionais

Quadro 5 — Estado dos motores e componentes de proteção no checkpoint

Tabela 1 — Estado atual das funcionalidades do CyberSafe Assistant

Quadro 6 — Estratégia de testes e níveis de evidência

Quadro 7 — Registo de riscos técnicos e controlos

Quadro 8 — Objetivos do projeto e evidência disponível

Figura C1 — Resultado da Análise Rápida do Computador

Figura C2 — Estado da Proteção e cobertura dos motores

Figura C3 — Validação end-to-end com canário benigno

LISTA DE ABREVIATURAS

Sigla	Significado
ACPI	Advanced Configuration and Power Interface
AMSI	Antimalware Scan Interface
API	Application Programming Interface
CRA	Cyber Resilience Act / Regulamento de Ciber-Resiliência
CPU	Central Processing Unit
CSF	Cybersecurity Framework
DNS	Domain Name System
EDR	Endpoint Detection and Response
ETW	Event Tracing for Windows
GDPR/RGPD	General Data Protection Regulation / Regulamento Geral sobre a Proteção de Dados
GPU	Graphics Processing Unit
HTTP/HTTPS	Hypertext Transfer Protocol / Secure
IA	Inteligência Artificial
IPC	Inter-Process Communication
JWT	JSON Web Token
NIST	National Institute of Standards and Technology
PAP	Prova de Aptidão Profissional
PC	Personal Computer
PE	Portable Executable
RAM	Random Access Memory
RPC	Remote Procedure Call
SBOM	Software Bill of Materials
SHA-256	Secure Hash Algorithm 256-bit
SSDF	Secure Software Development Framework
SVG	Scalable Vector Graphics

WFP	Windows Filtering Platform
WMI	Windows Management Instrumentation
YARA	Yet Another Recursive Acronym

1. INTRODUÇÃO

O presente relatório descreve o desenvolvimento do CyberSafe Assistant no âmbito da Prova de Aptidão Profissional do Curso Profissional de Programador/a de Informática da Escola Profissional de Gaia. A aplicação encontra-se numa fase avançada de desenvolvimento e esta versão reúne as decisões de arquitetura, o estado técnico comprovado, os testes, as dificuldades, as limitações e o trabalho futuro, complementados por um enquadramento teórico e regulamentar mais completo.

A ideia surgiu no 11.º ano com o objetivo de criar um antivírus relativamente simples. O desenvolvimento mostrou, contudo, que a segurança percebida por um utilizador não depende apenas de encontrar ou não um ficheiro malicioso. Também depende de saber se os motores estão disponíveis, de compreender a cobertura real de cada mecanismo, de observar a saúde do sistema, de distinguir pressão de recursos de uma ameaça e de receber uma orientação clara sobre o que fazer a seguir.

1.1 Contexto, problema e proposta de valor

O Windows disponibiliza uma grande quantidade de dados sobre processos, memória, discos, serviços, rede, firewall e eventos. Programas de segurança acrescentam ainda alertas, quarentena, assinaturas e resultados de análise. O problema identificado no projeto é que estes dados surgem frequentemente distribuídos por ferramentas diferentes e, para um utilizador com poucos conhecimentos técnicos, nem sempre explicam o significado prático de uma situação.

Desta necessidade nasceu o conceito atual: CyberSafe Assistant — Centro Inteligente de Segurança e Saúde do PC. A proposta de valor pode ser resumida pela ideia de que o computador produz sinais técnicos e o CyberSafe procura transformá-los em significado, prioridade e orientação, mantendo os detalhes disponíveis quando são necessários.

A experiência é estruturada em quatro passos de comunicação: DADO TÉCNICO → SIGNIFICADO → PRIORIDADE → ORIENTAÇÃO. O objetivo é que o utilizador consiga responder a perguntas simples — "O meu computador está bem?", "Existe algum problema?", "É grave?" e "O que devo fazer primeiro?" — sem que a interface esconda as limitações técnicas ou transforme incerteza em certeza.

1.2 Âmbito, limites e questão orientadora

O projeto não pretende substituir um antivírus comercial, um EDR empresarial ou uma plataforma forense. O âmbito da PAP é demonstrar uma arquitetura modular capaz de integrar mecanismos de análise, observação do Windows, saúde dos componentes de proteção, telemetria e assistência contextual, apresentando o estado real de forma compreensível e auditável.

A questão orientadora desta versão pode ser formulada da seguinte forma: como pode uma aplicação desktop reunir sinais de segurança e saúde do Windows, manter uma representação honesta da cobertura técnica e, ao mesmo tempo, explicar ao utilizador qual é a prioridade e a ação mais adequada?

> **Delimitação —a existência de um motor no código não é suficiente para afirmar proteção. O relatório diferencia implementação, disponibilidade em runtime, cobertura, validação e maturidade para distribuição.**

1.3 Organização do relatório

O capítulo 2 apresenta o enquadramento teórico e relaciona o projeto com gestão de risco, deteção de malware, defesa em profundidade, monitorização, usable security, inteligência artificial, privacidade, desenvolvimento seguro e legislação europeia. O capítulo 3 descreve a evolução e a implementação do CyberSafe, incluindo arquitetura, módulos, testes, dificuldades, riscos e trabalho futuro. O capítulo 4 discute criticamente os resultados face aos objetivos. O capítulo 5 apresenta a conclusão. O capítulo 6 reúne as referências bibliográficas, seguindo-se os anexos com evidências, quadros técnicos e capturas de ecrã.

2. ENQUADRAMENTO TEÓRICO

Este capítulo apresenta os conceitos que sustentam as principais decisões do CyberSafe Assistant. A fundamentação combina normas e documentação oficial com literatura científica e técnica. O objetivo não é transformar a PAP num tratado de cibersegurança, mas justificar por que razão o projeto combina mecanismos diferentes, evita garantias absolutas, trata a cobertura como variável explícita e dá importância à interpretação humana dos resultados.

2.1 Cibersegurança e gestão de risco

A cibersegurança deve ser entendida como um processo contínuo de gestão de risco e não como uma função isolada. O NIST Cybersecurity Framework 2.0 organiza resultados de alto nível nas funções Govern, Identify, Protect, Detect, Respond e Recover (NIST, 2024). O CyberSafe não implementa o CSF como norma de conformidade, mas o ciclo conceptual do projeto — monitorizar, detetar, priorizar, proteger, corrigir e verificar — apresenta uma relação clara com a ideia de observar o sistema, reduzir risco, responder e voltar a medir.

Esta perspetiva explica uma regra central: ausência de deteções não equivale a confirmação de segurança. Se um motor estiver indisponível, se uma base de assinaturas estiver vazia ou se determinado dado não puder ser observado, o nível de confiança deve diminuir. Por esse motivo, ProtectionHealth apresenta estados explícitos e o PcState utiliza findings para indicar aquilo que merece atenção.

Função CSF 2.0	Relação conceptual no CyberSafe	Exemplo no projeto
Govern	Regras, prioridades, limites e documentação de risco	Estados explícitos, organização por evidência, política de não esconder limitações
Identify	Conhecer ativos, estado e dependências	PC Center, processos, hardware, serviços, motores e cobertura
Protect	Reduzir exposição e aplicar controlos	Quarentena, proteção de processos internos, Windows Firewall, futuras políticas
Detect	Identificar sinais e eventos relevantes	Scanner, YARA-X, Hash DB, ClamAV quando disponível, Native Engine, findings
Respond	Orientar e executar ações perante um problema	Alertas, término seguro de processo, isolamento, Matrix contextual
Recover	Restabelecer estado e verificar resultado	Restauro de quarentena, revalidação de componentes e ciclo de verificação

Quadro 1 — Mapeamento conceptual entre NIST CSF 2.0 e CyberSafe. Fonte: elaboração própria com base em NIST (2024).

2.2 Malware e mecanismos de deteção

A deteção de malware pode recorrer a assinaturas, hashes, regras de padrões, análise estática, análise dinâmica, heurísticas e abordagens híbridas. A literatura distingue estas famílias precisamente porque cada uma observa propriedades diferentes e possui limitações próprias. Métodos baseados em assinaturas e hashes são eficazes para amostras conhecidas, mas dependem da atualização e da proveniência das bases; regras podem capturar características mais gerais, mas exigem boa qualidade para evitar falsos positivos; a análise dinâmica oferece contexto comportamental, mas aumenta a complexidade e o custo de execução (Souri & Hosseini, 2018; Gibert, Mateu & Planes, 2020).

No CyberSafe, esta complementaridade é representada pela base de hashes SHA-256, por regras YARA-X, pelo ClamAV, por heurísticas e pelo Native Engine. A documentação oficial do YARA-X descreve regras compostas por padrões e condições lógicas; o seu valor depende, por isso, do conteúdo do corpus e não apenas do facto de o motor estar operacional. Um YARA-X ativo com uma única regra comprova integração técnica, não cobertura abrangente.

O caso do falso positivo do Bun demonstrou ainda que combinar mais sinais não significa automaticamente produzir uma decisão melhor. Uma heurística genérica, uma regra pouco específica e um estado de assinatura desconhecido podem amplificar-se de forma incorreta. A qualidade da deteção exige ponderação, corroboração e testes de regressão.

2.3 Proteção em camadas, cobertura e confiança

A proteção em camadas reduz a dependência de um único mecanismo, mas cria uma responsabilidade adicional: o sistema precisa de conhecer a saúde e a cobertura de cada camada. Um componente pode estar tecnicamente ativo e, ao mesmo tempo, ter alcance limitado. Esta distinção é particularmente relevante para o estado atual do YARA-X e da Hash DB do CyberSafe.

O NIST SP 800-83 Rev. 1 recomenda combinar prevenção, manutenção dos mecanismos de proteção, deteção e preparação para resposta, em vez de depender de uma única tecnologia. No CyberSafe, este princípio é traduzido numa separação entre "estado do motor" e "cobertura conhecida". Assim, Ativo significa operacional nas condições verificadas; não significa proteção total.

O Protection Score deve, por isso, ser lido como indicador interno de confiança nos mecanismos verificados e não como probabilidade matemática de o computador estar seguro. Esta diferença é repetida na interface e no relatório para evitar que um número agregador esconda limitações importantes.

2.4 Monitorização de endpoint, telemetria e resposta a incidentes

Monitorizar um endpoint significa recolher sinais capazes de explicar o seu estado: utilização de CPU e memória, discos, rede, processos, serviços, eventos e alterações relevantes. Estes dados tornam-se úteis quando podem ser correlacionados, temporalmente contextualizados e ligados a uma ação. A NIST SP 800-61 Rev. 3, publicada em 2025, integra a resposta a incidentes no ciclo mais amplo de gestão de risco e reforça a preparação, deteção, resposta e recuperação como atividades conectadas (Nelson et al., 2025).

A investigação sobre EDR mostra também que logs e telemetria de endpoint podem tornar-se volumosos, o que cria problemas de armazenamento, desempenho e seleção de evidência. Para uma PAP desktop, a estratégia do CyberSafe é deliberadamente mais simples: um histórico curto em memória, findings priorizados e eventos suficientes para explicar o estado recente, evitando criar uma base permanente de telemetria sem necessidade.

Esta opção reduz complexidade e risco de privacidade, mas limita a análise histórica. A decisão é coerente com o âmbito do projeto: disponibilizar contexto imediato e verificável, não construir uma plataforma de retenção de logs empresarial.

2.5 Segurança utilizável e o utilizador no circuito

Uma ferramenta de segurança pode falhar do ponto de vista prático mesmo quando a deteção técnica está correta, caso o utilizador não compreenda o alerta ou não saiba o que fazer. Cranor (2008) propõe analisar explicitamente o papel do utilizador nos sistemas de segurança, considerando comunicação, comportamento esperado e capacidade de decisão. Trabalhos mais recentes continuam a mostrar que segurança e usabilidade precisam de ser desenhadas em conjunto, em vez de se assumir que a clareza aparecerá automaticamente (Theofanos, 2020; Furnell, 2024).

Uma revisão sistemática de 55 estudos sobre usable security identificou a necessidade de integrar requisitos de usabilidade desde fases iniciais de desenvolvimento (Di Nocera, Tempestini & Orsini,

2023). Em 2026, Di Nocera reforça uma perspetiva comportamental: o objetivo não é apenas tornar a interface agradável, mas reduzir situações em que o utilizador ignora, contorna ou interpreta incorretamente um mecanismo de segurança.

A resposta adotada no CyberSafe é a estrutura DADO TÉCNICO → SIGNIFICADO → PRIORIDADE → ORIENTAÇÃO. O dado original continua disponível, mas a camada principal procura responder primeiro ao impacto. Esta escolha é visível na tradução de estados técnicos, na distinção entre WFP e Windows Firewall e na preferência por "Indisponível" quando um sensor não devolve uma leitura fiável.

2.6 Inteligência artificial aplicada à assistência

A inteligência artificial pode ajudar a transformar dados técnicos em explicações mais acessíveis, mas também pode introduzir respostas não fundamentadas. O NIST AI Risk Management Framework 1.0 e o Generative AI Profile (NIST AI 600-1) tratam confiança, contexto, medição, governança e limitações como aspetos essenciais de sistemas de IA. O CyberSafe aplica esta preocupação de forma prática: a Matrix não deve ser tratada como fonte autónoma sobre o estado do computador.

A direção definida é de grounding no PcState, ProtectionHealth, findings e resultados concretos de operações. Quando não existe informação suficiente, a resposta correta pode ser pedir contexto ou afirmar que determinado dado não está disponível. A assistente deve ainda distinguir observações, inferências e recomendações, evitando converter "desconhecido" numa conclusão conveniente.

A utilização de IA no desenvolvimento do projeto segue a mesma lógica: ferramentas podem sugerir código, hipóteses e alternativas, mas as alterações precisam de integração, teste e validação. A autoria e a responsabilidade pela versão final do projeto permanecem do aluno.

2.7 Segurança em extensões de navegador

O módulo de navegador utiliza Manifest V3. A documentação do Chrome distingue permissões gerais, permissões opcionais e permissões de host, sendo que permissões mais invasivas podem produzir avisos explícitos ao utilizador. Esta arquitetura favorece a declaração clara das capacidades e apoia o princípio de menor privilégio.

No CyberSafe, a extensão foi concebida para observar sinais de risco relacionados com endereços, domínios, formulários e downloads, evitando recolher palavras-passe, cookies, histórico completo ou conteúdo privado sem necessidade. A integração final entre extensão e desktop permanece em validação, pelo que o relatório distingue o protótipo funcional de uma proteção de navegador totalmente operacional.

2.8 Privacidade, minimização e proteção de dados desde a conceção

O artigo 5.º do Regulamento (UE) 2016/679 estabelece, entre outros, os princípios de minimização, limitação da finalidade, exatidão e integridade/confidencialidade. O artigo 25.º reforça a proteção de dados desde a conceção e por defeito, exigindo que sejam tratados, por defeito, apenas os dados necessários para cada finalidade.

Estes princípios influenciam diretamente decisões do CyberSafe: a telemetria curta permanece em memória; a Matrix deve receber contexto sanitizado; logs não devem incluir palavras-passe, tokens ou outros segredos; e uma futura vertente Business deve privilegiar eventos e metadados estritamente necessários em vez de recolher conteúdo pessoal completo. Esta secção constitui enquadramento de desenho e não uma declaração formal de conformidade com o RGPD.

2.9 Windows Firewall e Windows Filtering Platform

Windows Firewall e Windows Filtering Platform estão relacionados, mas não são a mesma coisa. A Microsoft descreve a WFP como uma plataforma de APIs e serviços do sistema para criar aplicações de filtragem em vários pontos da pilha de rede. O Windows Firewall utiliza mecanismos do Windows para aplicar políticas; uma aplicação pode, por sua vez, criar filtros próprios sobre a infraestrutura WFP.

Esta distinção teve impacto direto no CyberSafe. Uma versão anterior utilizava a designação "firewall" para um estado que correspondia ao filtro WFP próprio do projeto. A correção separou os dois conceitos: o Windows Firewall pode estar ativo enquanto o filtro WFP do CyberSafe permanece indisponível. A interface atual procura tornar esta diferença visível, evitando transformar a falha de um componente próprio numa afirmação falsa sobre a proteção nativa do Windows.

2.10 Segurança de aplicações Electron e comunicação IPC

Electron combina tecnologias Web com capacidades desktop, o que exige uma fronteira clara entre o renderer e as APIs privilegiadas. A documentação oficial recomenda, entre outras medidas, context isolation, sandboxing, limitação de navegação, política de conteúdo restritiva, validação do remetente de mensagens IPC e não exposição de APIs Electron a conteúdo não confiável.

A arquitetura do CyberSafe segue uma separação por camadas: renderer React → preload/Electron → IPC → serviço local → named pipe/RPC → core e integrações com o Windows. Esta abordagem reduz o acesso direto da interface a operações privilegiadas e facilita testes isolados. Em contrapartida, aumenta a importância dos contratos: um campo interpretado incorretamente entre camadas pode produzir um estado visual errado apesar de o backend funcionar.

> **Ponto de segurança a verificar antes da distribuição —a existência da fronteira arquitetural não substitui uma auditoria das opções concretas de Electron. Context isolation, sandboxing, Content Security Policy, validação do sender IPC e restrição de navegação devem ser confirmados na build final.**

2.11 Logging, rastreabilidade e qualidade da evidência

Logs são úteis para compreender falhas, provar comportamentos e investigar incidentes, mas também podem criar risco. O OWASP Logging Cheat Sheet recomenda registar eventos relevantes de forma consistente e evitar a gravação direta de segredos, tokens, palavras-passe e dados pessoais desnecessários. A qualidade do log depende tanto do que é registado como da possibilidade de ligar o evento ao contexto correto.

No CyberSafe, logs e probes são utilizados como evidência de runtime, mas o relatório não trata uma mensagem de log isolada como prova suficiente quando o comportamento depende da interface. Fluxos como término de processos, scanner e estados dos motores exigem validação end-to-end, porque o resultado que chega ao utilizador é a combinação de várias camadas.

2.12 Desenvolvimento seguro e cadeia de fornecimento de software

O NIST Secure Software Development Framework (SSDF) propõe práticas de desenvolvimento seguro que podem ser integradas em diferentes ciclos de vida. Entre os objetivos encontram-se preparar a organização, proteger o software, produzir software mais seguro e responder a vulnerabilidades. Para o CyberSafe, esta perspetiva é relevante porque o projeto combina dependências npm, Electron/Chromium/Node.js, bibliotecas, motores externos, scripts, componentes nativos e assets.

A cadeia de fornecimento não é apenas uma questão de atualizar pacotes. Inclui proveniência, licença, integridade e compatibilidade dos componentes. O próprio projeto já encontrou exemplos concretos: regras YARA precisam de origem e licença conhecidas; hashes locais precisam de proveniência verificável; um Native Engine empacotado ficou desatualizado após a correção SHA-256; e a remoção de uma dependência de baixo nível do helper de temperaturas mostrou que "funcionar" não é critério suficiente para distribuição.

Área	Risco de desenvolvimento	Controlo aplicado ou recomendado
Dependências	Versões vulneráveis ou componentes abandonados	Inventário de dependências, atualização controlada, revisão de advisories e lockfiles
Motores/regras	Fonte incerta, licença inadequada ou cobertura mal interpretada	Proveniência, licença, testes de regressão, cobertura explícita
Binários nativos	Build antiga incluída por engano	Rebuild controlado, checksum, smoke test do pacote e separação DEV/release
IPC/named pipe	Operação privilegiada acionada por origem indevida	Contratos tipados, validação de sender/origem, autenticação local quando aplicável
Logs	Exposição de tokens, caminhos ou dados pessoais	Sanitização, minimização, rotação e exclusão de segredos
Git/release	Perda de alterações ou estado remoto desatualizado	Commits pequenos, revisão de diff, tags/checkpoints e sincronização deliberada

Quadro 4 — Desenvolvimento seguro, privacidade e rastreabilidade aplicados ao projeto. Fonte: elaboração própria, com base em NIST SSDF e OWASP.

2.13 Regulamento de Ciber-Resiliência: contexto europeu

O Regulamento (UE) 2024/2847, conhecido como Cyber Resilience Act (CRA), estabelece requisitos horizontais de cibersegurança para produtos com elementos digitais colocados no mercado europeu. Entre os princípios do Anexo I encontram-se segurança baseada no risco, configuração segura por

defeito, correção de vulnerabilidades, proteção contra acesso não autorizado, confidencialidade de dados e tratamento documentado de vulnerabilidades.

O regulamento é especialmente relevante como referência conceptual porque o Anexo III inclui, na classe I, "software que pesquisa, remove ou coloca em quarentena software malicioso". Se o CyberSafe viesse a ser comercializado na União Europeia com estas funções como núcleo do produto, essa categoria poderia tornar-se juridicamente relevante. No entanto, esta PAP não é um produto colocado no mercado, e o relatório não declara classificação, conformidade ou certificação.

A aplicação geral do CRA começa em 11 de dezembro de 2027, existindo algumas disposições com aplicação anterior. Para o projeto, o valor imediato está em adotar desde já práticas coerentes com o espírito do regulamento: documentar risco, conhecer componentes, manter atualizações, tratar vulnerabilidades e não esconder limitações conhecidas.

Nota jurídica —o enquadramento do CRA é informativo e académico. A eventual classificação de uma versão comercial dependeria do produto efetivamente colocado no mercado, da sua finalidade, funcionalidades e cadeia de distribuição.

2.14 Sobreposições entre segurança, saúde, desempenho, privacidade e assistência

O CyberSafe não trata segurança, desempenho e assistência como módulos isolados. A mesma evidência pode ter significados diferentes consoante o contexto. Um disco quase cheio é, à partida, um problema de sistema, mas pode impedir atualizações, logs ou quarentena. Um processo com CPU elevada pode ser apenas uma carga legítima; o mesmo processo, se for desconhecido e estabelecer ligações anómalas, passa a exigir análise de segurança. Uma ligação de rede pode ser normal em desempenho, relevante em privacidade e suspeita em segurança, dependendo do processo, destino, protocolo e histórico.

Esta sobreposição justifica o PcState como camada de integração. Em vez de cada página inventar a sua conclusão, o projeto procura reutilizar evidências e findings comuns. ProtectionHealth acrescenta a dimensão de confiança nos mecanismos de proteção; a Matrix acrescenta linguagem e orientação; nenhuma destas camadas substitui os dados originais.

Figura 3 — Mapa conceptual de sobreposições funcionais no CyberSafe. Fonte: elaboração própria.

Origem do dado	Interpretação	Sobreposição principal	Saída adequada para o utilizador
Motor de proteção degradado	Menor confiança na cobertura	Segurança + estado do sistema	Finding de segurança e orientação para corrigir o motor
CPU/RAM/disco elevados	Pressão de recursos	Desempenho + saúde	Finding de desempenho/sistema; não é ameaça por si só
Processo desconhecido	Necessita de contexto	Processos + segurança	Dados do processo, assinatura/confiança e ação segura
Processo com alto consumo e rede	Pode ser carga legítima ou sinal a investigar	Desempenho + processos + rede + segurança	Correlacionar consumo, origem, assinatura e destino antes de elevar risco
Ligação de rede nova	Evidência a contextualizar	Rede + segurança + privacidade	Destino/processo/protocolo e prioridade apenas se existirem sinais adicionais
Pouco espaço em disco	Risco operacional	Saúde + segurança	Explicar impacto em updates, quarentena e estabilidade; sugerir libertação segura
Resultado de scan sem deteções	Nenhuma deteção dentro da cobertura usada	Segurança + confiança	Resultado da análise sem afirmar segurança absoluta
Temperatura não lida	Dado desconhecido	Saúde + confiança	Mostrar "Indisponível"; não estimar um valor
Finding ou estado técnico	Contexto estruturado	Assistência + todas as áreas	Matrix explica significado, prioridade e primeira ação
Log técnico com erro	Evidência de diagnóstico	Operação + segurança + privacidade	Apresentar resumo ao utilizador; manter detalhes sanitizados em diagnóstico

Quadro 3 — Sobreposições funcionais entre dados, interpretação, risco e orientação no CyberSafe. Fonte: elaboração própria.

2.15 Síntese teórica e artigos relacionados

A revisão teórica sustenta quatro decisões centrais do produto. Primeiro, a segurança é um processo e não um estado binário. Segundo, mecanismos diferentes devem ser combinados sem esconder a cobertura limitada de cada um. Terceiro, a apresentação ao utilizador faz parte da eficácia da segurança. Quarto, o próprio desenvolvimento da aplicação precisa de incorporar privacidade, rastreabilidade e controlo da cadeia de fornecimento.

Referência	Tema	Contributo para o CyberSafe
Cranor (2008)	Humano no circuito de segurança	Fundamenta a necessidade de considerar a decisão do utilizador como parte do sistema.
Souri & Hosseini (2018)	Deteção de malware	Reforça vantagens e limitações de assinaturas, comportamento e métodos híbridos.
Gibert, Mateu & Planes (2020)	Deteção/classificação de malware	Organiza abordagens estáticas, dinâmicas e híbridas e evidencia desafios de generalização.
Theofanos (2020)	Usable security	Defende que medidas de segurança dependem da participação ativa do utilizador.
Di Nocera et al. (2023)	Revisão sistemática de usable security	Mostra a importância de requisitos de usabilidade desde o desenvolvimento.
Furnell (2024)	Usable cybersecurity	Salienta que segurança utilizável precisa de ser concebida por design.
Di Nocera (2026)	Usable security e comportamento	Propõe olhar para comportamentos e contorno de medidas, não apenas para estética/interface.
NIST SP 800-61 Rev. 3 (2025)	Resposta a incidentes	Integra resposta no ciclo de gestão de risco e reforça deteção, resposta e recuperação.
NIST SSDF (2022)	Desenvolvimento seguro	Apoia controlo de dependências, práticas de release, resposta a vulnerabilidades e evidência.
NIST AI 600-1 (2024)	Risco de IA generativa	Apoia grounding, gestão de limitações e comunicação de incerteza na Matrix.

Quadro 2 — Literatura e artigos técnicos utilizados no enquadramento. Fonte: elaboração própria.

3. DESENVOLVIMENTO DO PROJETO

Este capítulo apresenta o percurso de desenvolvimento do CyberSafe Assistant. São descritas a evolução da ideia, a organização do trabalho, as tecnologias utilizadas, a arquitetura e os principais módulos. Sempre que uma funcionalidade não se encontra totalmente concluída, o respetivo estado é indicado de forma explícita, para que o relatório represente o projeto real e não uma versão idealizada.

3.1 Apresentação e evolução da ideia

A ideia do CyberSafe surgiu quando frequentava o 11.º ano. Nessa fase, pretendia desenvolver um antivírus funcional, inspirado em soluções conhecidas, capaz de analisar ficheiros e contribuir para a proteção do computador. Para diferenciar o projeto, procurei acrescentar funcionalidades que não fossem apenas uma reprodução visual de um antivírus tradicional.

Durante o desenvolvimento, percebi que a informação de segurança se encontra frequentemente separada de outros dados importantes do sistema. Um utilizador pode ter uma proteção ativa e, ainda assim, não compreender o estado do firewall, os processos, o consumo de memória, o espaço em disco ou a disponibilidade dos motores. O projeto evoluiu, assim, de antivírus simples para plataforma de segurança, depois para saúde do computador e assistência inteligente, e mais recentemente para uma primeira fase Business.

Figura 1 — Ciclo conceptual do CyberSafe Assistant. Nem todas as etapas se encontram automatizadas. Fonte: elaboração própria.

3.2 Motivação e fundamentação

A escolha do tema está relacionada com o meu interesse por programação, informática e cibersegurança. Pretendia desenvolver um projeto real, com dimensão suficiente para exigir aprendizagem contínua, integração entre tecnologias e resolução de problemas que não fossem apenas exercícios isolados.

Outra motivação foi compreender melhor como funciona a proteção de um computador. A criação do projeto obrigou-me a investigar análise de ficheiros, estados de motores, processos, serviços do Windows, comunicação entre aplicações, rede, hardware e formas de apresentar resultados. Também identifiquei uma dificuldade prática: percentagens, PIDs, portas, hashes e nomes de serviços são úteis para utilizadores avançados, mas não respondem diretamente às dúvidas de uma pessoa com menos experiência.

3.3 Objetivos

3.3.1 Objetivo geral

Desenvolver uma aplicação desktop modular capaz de reunir informação sobre segurança e saúde do computador, apresentar o estado do sistema de forma compreensível e orientar o utilizador sem criar uma falsa sensação de proteção.

3.3.2 Objetivos específicos

- criar uma interface desktop adequada a utilizadores comuns de Windows;
- implementar mecanismos de análise de ficheiros e representação do estado dos motores;
- distinguir a ausência de deteções da confirmação de que o computador está seguro;
- monitorizar dados reais de processador, memória, disco, placa gráfica, rede e processos;
- transformar informação técnica em findings com categoria e prioridade;
- disponibilizar quarentena e gestão dos elementos isolados;
- integrar uma assistente capaz de explicar resultados de forma curta e contextualizada;
- utilizar uma arquitetura modular que permita acrescentar ou substituir componentes;
- testar funcionalidades e registar de forma transparente limitações e regressões;

• explorar autenticação, extensão de navegador e vertente Business sem apresentar protótipos como produto concluído;

• aplicar princípios de desenvolvimento seguro, minimização de dados e rastreabilidade compatíveis com a evolução futura do projeto.

3.3.3 Evolução dos objetivos

Os objetivos não permaneceram inalterados. A proteção contra malware passou a ser apenas uma das dimensões do produto. O feedback recebido, especialmente do professor orientador, levou a reforçar saúde do PC, clareza de estados, Business Security Center, autenticação e documentação das limitações. Ferramentas como Nmap e Wireshark foram estudadas como referências conceptuais para descoberta e análise de rede, mas não se encontram integradas no CyberSafe.

3.4 Público-alvo

O público-alvo principal é constituído por utilizadores comuns de Windows que pretendem perceber o estado do computador sem dominar termos técnicos de segurança ou administração de sistemas. Utilizadores avançados podem beneficiar de detalhes sobre processos, ligações, motores, eventos e hashes, mas essa densidade técnica não deve prejudicar a camada principal.

Foi ainda identificado um possível público empresarial futuro, sobretudo pequenas organizações sem equipa dedicada de cibersegurança. Esta vertente permanece delimitada ao que já está implementado no endpoint; inventário de rede, análise avançada de tráfego, políticas multi-endpoint e alertas centralizados são trabalho futuro.

3.5 Planeamento e metodologia

3.5.1 Desenvolvimento individual e apoio externo

O CyberSafe Assistant foi desenvolvido individualmente por mim, Caio Cesar Silva Amorim. A conceção do produto, as decisões de arquitetura, a integração dos componentes, a validação e a evolução permaneceram sob a minha responsabilidade. Professores, amigos e outras pessoas contribuíram com orientação, sugestões e testes, mas estes contributos devem ser distinguidos da autoria do desenvolvimento.

3.5.2 Desenvolvimento iterativo orientado por evidência

O trabalho decorreu de forma progressiva e iterativa. Funcionalidades eram planeadas, implementadas, testadas e ajustadas. Quando a validação mostrava instabilidade, o estado regressava a "em validação" ou "parcial", em vez de ser mantido artificialmente como concluído. A metodologia tornou-se particularmente importante num sistema com várias camadas: um sintoma na interface pode ter origem no preload, IPC, ServiceClient, named pipe, serviço, core ou integração com o Windows.

3.5.3 Níveis de maturidade e definição de concluído

Para reduzir ambiguidades, esta versão do relatório separa quatro níveis de evidência. "Implementado" significa que existe código utilizável no âmbito descrito. "Integrado" significa que a função atravessa as camadas necessárias. "Validado em runtime" significa que o comportamento foi observado no ambiente real, manualmente ou por probe adequado. "Pronto para distribuição" exige ainda packaging, configuração, dependências, segurança e smoke tests da build. Uma funcionalidade pode cumprir os três primeiros níveis e ainda não estar pronta para distribuição.

Nível	Pergunta de controlo	Exemplo
Implementado	Existe código funcional para o comportamento?	Scanner possui estados, modos e lógica de serviço.
Integrado	O fluxo atravessa as camadas necessárias?	Evento sai do serviço e chega ao Electron/preload/UI.
Validado em runtime	Foi observado no Windows real com resultado esperado?	Notepad terminado por PID e mensagem confirmada.
Regressão protegida	Existe teste/probe que deteta a reintrodução do erro?	Bun deixa de ser crítico e EICAR mantém comportamento controlado.
Pronto para distribuição	A build empacotada contém os binários/configs corretos e passou smoke test?	Ainda pendente para Native Engine corrigido e alguns módulos.

Quadro 5 — Níveis de evidência utilizados para interpretar o estado das funcionalidades. Fonte: elaboração própria.

3.5.4 Git e GitHub

O projeto utiliza Git e possui um repositório público em GitHub. O controlo de versões permite acompanhar alterações, comparar estados e recuperar trabalho. Durante o desenvolvimento ocorreu uma situação em que um comando repôs um ficheiro do scanner para a versão registada, removendo alterações locais ainda não commitadas. O incidente reforçou a necessidade de commits menores, revisão do estado da working tree e backups antes de operações destrutivas.

O repositório público encontra-se em https://github.com/MatrixChill/cybersafe-assistant. Durante o desenvolvimento existiram períodos em que a working tree local esteve à frente do remoto; por esse motivo, a data de um commit publicado não deve ser usada isoladamente como prova do estado completo do projeto.

3.5.5 Utilização de inteligência artificial e Maestri

A principal ferramenta utilizada para apoiar o desenvolvimento foi o Matrix Code, criado por mim. Foram também utilizadas, em diferentes momentos, ferramentas como Antigravity, Visual Studio, Codex, Claude, v0.app e Cursor. O v0.app foi usado como laboratório visual, não como substituto do backend ou dos dados reais.

Nas fases mais recentes, o Maestri foi usado como infraestrutura de coordenação multiagente. O princípio adotado foi separar papéis — coordenação, frontend, backend/Windows, revisão e testes mecânicos — atribuir um único responsável de escrita por tarefa e exigir validação independente. Um pedido enviado a um agente não é considerado execução até existir evidência de início e resultado. Operações Git destrutivas foram restringidas e fluxos dependentes de runtime mantiveram validação humana obrigatória.

3.5.6 Organização, rastreabilidade e documentação

As tarefas são acompanhadas num organizador próprio da PAP, onde cada item é classificado segundo evidência. No checkpoint de 30 de setembro de 2026 existiam 206 tarefas: 102 concluídas, 27 em curso e 77 planeadas. Pelo método interno do projeto, o roadmap encontrava-se em 56,1% e o subconjunto prioritário da PAP em 77,7%. Estas percentagens são indicadores de planeamento interno, não métricas externas de qualidade.

3.6 Ferramentas e tecnologias utilizadas

TypeScript é a linguagem principal de grande parte do projeto e permite partilhar tipos entre packages. React e Vite suportam a interface; Electron disponibiliza a camada desktop; Node.js é utilizado na aplicação, serviço e componentes auxiliares; PowerShell apoia integrações com o Windows; C++ é utilizado no Native Engine; SQLite, bcrypt e JWT suportam a autenticação; Manifest V3 é utilizado na extensão.

Tecnologia	Papel no projeto	Observações
TypeScript	Lógica principal e contratos partilhados	Reduz diferenças de tipos entre app, desktop, service e core.
React + Vite	Interface e build do renderer	Componentização, estado de páginas e desenvolvimento rápido.
Electron	Aplicação desktop e fronteira privilegiada	Preload, IPC, lifecycle e integração com o serviço local.
Node.js	Serviço, scripts e análise	Orquestra operações e integrações de alto nível.
PowerShell	Consultas e operações Windows	Serviços, firewall, processos, discos e outros dados do sistema.
YARA-X	Regras de padrões	Motor ativo com cobertura atualmente limitada.
Hash DB / SHA-256	Correspondência por digest	Base local pequena; proveniência ainda em limpeza.
ClamAV	Motor independente de assinaturas	Instalado, mas degradado por ausência de base.
C++ Native Engine	Hash, entropia e parsing PE	Operacional em DEV; packaging precisa de rebuild controlado.
SQLite + bcrypt + JWT	Persistência e autenticação	Conta, sessão e role de autorização.
Git/GitHub	Controlo de versões	Estado remoto pode ficar temporariamente atrás da working tree.

Quadro 6 — Principais tecnologias e responsabilidades no CyberSafe Assistant. Fonte: elaboração própria.

3.7 Arquitetura do sistema

O CyberSafe encontra-se organizado como monorepo. Esta estrutura mantém diferentes componentes no mesmo repositório, permite partilhar tipos e coordena versões sem colocar todas as responsabilidades numa única aplicação. Os packages principais incluem app, desktop, service, core, api, shared-types, native-windows, native-engine e extension.

Figura 2 — Arquitetura lógica e fronteiras de confiança do CyberSafe Assistant. Fonte: elaboração própria.

3.7.1 Fluxo principal

Um fluxo típico segue a sequência: ação do utilizador → função exposta pelo preload → IPC no processo principal → pedido RPC pelo named pipe → serviço/core → integração com Windows ou motor → resposta/evento → atualização da interface. A separação melhora organização e testabilidade, mas torna os contratos e a ordem temporal dos eventos críticos.

3.7.2 Packages e responsabilidades

O package packages/app contém a interface React; packages/desktop gere Electron, preload e ligação ao serviço; packages/service recebe pedidos e publica eventos; packages/core concentra análise, monitorização, PcState e ProtectionHealth; packages/api suporta autenticação e serviços associados; packages/shared-types centraliza contratos; packages/native-windows contém integrações específicas do Windows; packages/native-engine reúne código C++; packages/extension contém a extensão de navegador.

3.7.3 Fronteiras de confiança

A interface não deve executar diretamente operações privilegiadas. O renderer é tratado como camada de apresentação; o preload expõe apenas a superfície necessária; o processo principal valida e traduz pedidos; o serviço local mantém operações de sistema e motores. Antes da distribuição final, estas fronteiras deverão ser auditadas face às recomendações de segurança do Electron, em especial sender validation, context isolation, sandboxing e navegação restrita.

3.8 Desenvolvimento da interface

3.8.1 Estrutura de navegação e Dashboard

O Dashboard procura responder rapidamente a "Como está o meu computador?". Reúne estado geral, score de proteção, categorias do PcState, finding principal, Matrix, métricas, eventos, ameaças, quarentena e ações rápidas. A barra lateral organiza o acesso às restantes áreas. Pequenos gráficos de tendência foram criados com SVG para evitar uma biblioteca pesada apenas para séries curtas.

3.8.2 Onboarding

Foi desenvolvido um onboarding com três passos relacionados com proteção, scanner e Matrix. A conclusão ou omissão é guardada localmente e existe opção para reabrir a introdução nas Configurações. Foram acrescentados testes específicos para este comportamento.

3.8.3 Temas e identidade visual

O sistema visual separa modo de luminosidade — claro, escuro ou sistema — de tema e cor de destaque. Foram trabalhadas identidades CyberSafe Core, CyberSafe Neon, Matrix Terminal, CyberSafe Liquid e Red Belbete. As cores semânticas de sucesso, aviso e perigo permanecem independentes da identidade escolhida. Um problema de especificidade CSS que impedia alguns accents no modo claro foi corrigido através de seletores equivalentes e testes das combinações.

3.9 ProtectionHealth e PcState

3.9.1 ProtectionHealth

ProtectionHealth representa o estado real dos componentes de proteção. Os estados previstos incluem active, degraded, unavailable e error. "Active" significa disponível nas condições verificadas;

"degraded" indica funcionamento limitado ou fallback; "unavailable" significa que o componente não pode ser utilizado; "error" identifica uma falha que impede o comportamento esperado.

3.9.2 Motores e componentes de proteção

A passagem de qualidade mais recente separou o estado técnico do motor da cobertura efetiva. Quatro motores de análise são considerados no score principal: YARA-X, Hash DB, ClamAV e Native Engine. Windows Firewall e o filtro WFP próprio do CyberSafe são apresentados separadamente porque possuem responsabilidades diferentes.

Componente	Estado observado	Cobertura/evidência	Limitação principal
YARA-X	Ativo	1 regra carregada	Cobertura de regras muito limitada
Hash DB	Ativo	4 assinaturas locais	Proveniência e rotulagem ainda em limpeza
ClamAV	Degradado	Binário instalado	Base de assinaturas ausente; freshclam.conf é o primeiro bloqueio confirmado
Native Engine	Ativo em DEV	SHA-256, entropia e PE; hash corrigido	Binário empacotado anterior está desatualizado
Windows Firewall	Ativo na verificação direta	Perfis Domain, Private e Public	Leitura deve permanecer separada do WFP próprio
Filtro WFP do CyberSafe	Indisponível	WFP filter not started	Não equivale a Windows Firewall desligado

Quadro 7 — Estado dos motores e componentes no checkpoint técnico de 30/09/2026. Estados específicos do ambiente e do momento da medição.

3.9.3 Protection Score e PcState

Na validação visual de 30 de setembro de 2026, o Protection Score apresentou 80/100 porque três dos quatro motores de análise estavam ativos e um degradado. Este valor não representa "80% seguro". A mesma vista passou a mostrar a cobertura reduzida de YARA e Hash DB. O PcState representa a saúde global do computador nas categorias Segurança, Desempenho, Sistema e Privacidade, com severidades Urgente, Atenção, Recomendado e Informativo.

PcState e ProtectionHealth funcionam como fontes comuns para reduzir contradições entre Dashboard, PC Center, relatórios e Matrix. A integração do relatório interno com estas fontes ainda precisa de ser completada.

3.10 Monitorização e PC Center

3.10.1 Hardware, desempenho e sistema

O PC Center reúne hardware, desempenho, sensores, processos, rede e saúde do Windows. Os valores devem ter origem no sistema real. Uma percentagem de memória apresentada no Dashboard corresponde à memória global do computador, e não apenas ao consumo do CyberSafe; esta distinção foi importante durante a investigação de desempenho.

3.10.2 Telemetria

Foi implementado um ring buffer em memória com aproximadamente sessenta amostras de determinadas métricas, atualizado periodicamente. O histórico não é persistido e perde-se ao encerrar a aplicação. Esta janela curta permite observar tendências de CPU, memória, rede e GPU e reduzir findings provocados por picos momentâneos.

3.10.3 Temperaturas

A leitura de temperaturas permanece parcial. Foram investigadas systeminformation, WMI, ACPI Thermal Zone e LibreHardwareMonitor. No computador de desenvolvimento, os métodos WMI e ACPI não devolveram leitura fiável da CPU. Uma leitura de GPU surgiu temporariamente e desapareceu em atualizações seguintes, demonstrando a necessidade de identificar corretamente múltiplas GPUs e controlar o tempo de validade dos sensores.

Em vez de estimar um valor, a interface apresenta "Indisponível". Um helper experimental em .NET baseado em LibreHardwareMonitor e dependência de baixo nível foi retirado do caminho ativo por não cumprir o nível de segurança pretendido para distribuição. A arquitetura atual privilegia providers mais seguros e ausência explícita de leitura.

3.11 Scanner e quarentena

3.11.1 Arquitetura e estados do scanner

A área Análise de Segurança suporta Análise Rápida, Análise Profunda, ficheiro específico, pasta e inspeção de ZIP. Existem estados para preparação, identificação, scanning, finalização, conclusão, cancelamento e falha. A Análise Rápida usa áreas prioritárias, profundidade limitada e budget de candidatos; a Análise Profunda percorre scopes definidos e arranques automáticos até um limite máximo, procurando explicar esse âmbito ao utilizador sem prometer uma análise integral de todos os volumes.

O serviço foi testado diretamente pelo named pipe e através de probes que atravessam o fluxo real. Foram validados ficheiro único, pasta, pasta vazia e ZIP. Extensões de maior risco como .exe, .dll, .msi, .sys, .lnk, .ps1 e .zip deixaram de ser excluídas apenas pela extensão. Num scan de pasta, um ZIP externo conta como candidato principal e as entradas internas podem ser analisadas sem inflacionar artificialmente o contador visível.

3.11.2 Integração com Electron e validação em runtime

Foi corrigida uma condição de corrida em que o scan podia emitir progresso ou terminar antes de a interface registar o listener. O listener passou a ser estável, eventos precoces são guardados temporariamente e o serviço mantém a sessão como fonte de verdade. A interface consegue recuperar o estado ao regressar à página. Foi ainda alinhado o critério de threat.detected com threatsFound, evitando situações em que um alerta de ameaça coexistia com total final de zero ameaças.

Num teste da Análise Rápida foram processados 800 candidatos: 747 sem deteções, 53 ignorados por exclusões, zero ameaças e duração aproximada de 154 segundos. Estes valores correspondem apenas ao computador de desenvolvimento e não são um benchmark universal. Permanecem gates pequenos de QA: pasta pequena, ZIP pequeno, cancelamento, estados completos, timer, origem da deteção e copy. A Análise Profunda mantém trabalho de qualidade em percentagem, desempenho e alguns casos de ZIP aninhado.

3.11.3 Quarentena

A quarentena suporta mover, restaurar, eliminar e manter metadados. O isolamento pretende impedir a utilização normal do ficheiro e preservar informação para eventual recuperação. A quantidade de itens em quarentena não deve ser apresentada como quantidade de vírus, porque um elemento pode resultar de ação preventiva ou deteção que ainda necessita de confirmação.

3.11.4 Limite de recursos e orquestração do motor

Uma análise em lote real identificou um padrão de esgotamento de recursos: o arranque de múltiplas análises simultâneas provocava um crescimento em cascata da utilização de RAM e podia resultar na coexistência de vários processos clamscan.exe. Foi introduzido um tecto no número de processos clamscan.exe concorrentes e um limite de RAM em cascata no núcleo do motor ClamAV. Num reteste manual de 2026-10-03 a RAM estabilizou em torno de 60–62 %, com um pico de um único processo clamscan no Gestor de Tarefas. A presença de clamscan.exe durante a análise é o comportamento esperado de orquestração de motor externo, não um defeito de marcação. A eliminação do processo externo em favor de um motor nativo próprio permanece uma opção de arquitectura futura.

3.12 Matrix

3.12.1 Mascote e assistente

A Matrix funciona como identidade visual e como assistente de inteligência artificial. Existem vários assets e moods, incluindo safe, guide, alert, success, happy, defender, avatar e mini. A mascote contribui para a identidade do produto sem substituir o estado do computador como elemento principal da interface.

3.12.2 Política de resposta

As respostas devem explicar o que aconteceu, indicar gravidade e sugerir a primeira ação. Foram criados testes de política relacionados com brevidade, grounding, resposta ao caso concreto e

prevenção de explicações genéricas. Se não existir contexto suficiente para afirmar por que uma ligação é phishing, a Matrix deve pedir a ligação ou declarar a falta de dados, e não inventar uma causa.

A assistente também não deve afirmar que o computador está seguro quando os dados apenas indicam ausência de deteções. O contexto enviado deve ser sanitizado, evitando caminhos pessoais, endereços de correio eletrónico e outros dados desnecessários.

3.12.3 Evolução contextual

A direção futura é transformar a Matrix de chatbot isolado em assistente contextual do CyberSafe, capaz de explicar findings, componentes degradados e resultados de ações. Parte das políticas já está implementada; a integração contextual com todas as áreas continua em desenvolvimento.

3.13 Módulos complementares

3.13.1 Extensão de navegador

O package packages/extension contém manifest, service worker, content script e popup em Manifest V3. Existe lógica inicial para sinais de endereços suspeitos, domínios semelhantes, punycode, HTTP inseguro, formulários sensíveis em contexto suspeito e downloads. A extensão deve apresentar sinais e não afirmar roubo de dados sem evidência. Pairing e autenticação definitiva entre extensão e desktop permanecem por validar.

3.13.2 Autenticação e conta

A API inclui registo, login e consulta do utilizador autenticado, com bcrypt, JWT e SQLite. Em setembro de 2026 foi introduzido um campo de role user/admin separado do tipo Personal/Business e do plano. O registo público força role user; uma conta administrativa de desenvolvimento pode ser elevada apenas por fluxo restrito a DEV. A validação de formato do email não equivale a verificação do endereço, que permanece futura.

3.13.3 Preferências, notificações e autostart

Preferências de modo, tema, accent e onboarding são guardadas localmente. Sincronização cloud não está concluída. Notificações possuem infraestrutura inicial e alguns eventos; conclusão de scan, proteção degradada, extensão e outros cenários continuam em evolução. Autostart e arranque minimizado foram implementados, mas necessitam de validação final na versão empacotada.

3.13.4 Relatório interno

O CyberSafe possui uma página de relatórios com apresentação A4, branding, resumo, ameaças, quarentena, motores, hardware e findings. Uma versão inicial calculava um score próprio, o que podia divergir das restantes páginas. A direção correta é consumir PcState e ProtectionHealth como fontes centrais; esta integração ainda não está concluída em todas as secções.

3.13.5 Componentes nativos e tecnologias investigadas

O Native Engine em C++ calcula SHA-256, entropia e informação PE e está integrado no fluxo de desenvolvimento. ETW, AMSI e WFP foram investigados em protótipos ou simulações de espaço de utilizador. O projeto não possui atualmente um driver kernel completo desenvolvido pelo CyberSafe.

3.13.6 CyberSafe Business — fase inicial

Existem contas Personal e Business, controlo de acesso ao Business Security Center e dados reais do endpoint, como conexões e processos. A fase atual não constitui gestão empresarial multi-endpoint. Descoberta autorizada de rede, inventário, análise de tráfego, deteção de anomalias, políticas e alertas centralizados permanecem trabalho futuro.

3.14 Testes e validação

3.14.1 Estratégia de testes e níveis de evidência

A validação combina testes unitários e de integração por workspace, typecheck, builds, git diff --check, probes de runtime e testes manuais. Para fluxos dependentes do Windows, Electron ou hardware, uma suite verde não substitui observação em runtime. Os números de testes são checkpoints e devem ser executados novamente antes da entrega final.

Nas passagens de qualidade de setembro foram documentadas suites com diferentes contagens ao longo do tempo. Um checkpoint mais recente registou 253 testes no core, 385 na app e 160 no service, sem falhas, além de typecheck e git diff --check limpos; desktop e native tinham sido verificados anteriormente com 29 e 60 testes. Estes valores não são apresentados como garantia absoluta, mas como evidência do estado testado naquele momento.

Tipo de validação	O que prova	O que não prova sozinho	Exemplos no projeto
Teste unitário	Regra/função isolada responde ao caso previsto	Integração com Windows e UI	PcState, deduplicação, políticas da Matrix
Teste de integração	Contrato entre componentes	Comportamento visual completo	Scanner service/client, ressincronização
Typecheck/build	Coerência de tipos e compilação	Semântica correta em runtime	Monorepo e workspaces
Probe de runtime	Fluxo real sem depender de observação visual completa	Clareza da interface	Named pipe, scanner, engines
Teste manual end-to-end	Resultado técnico + experiência visível	Cobertura sistemática de todos os regressions	Notepad, canário benigno, proteção WS-B
Teste com utilizador	Compreensão, navegação e decisão	Correção interna do algoritmo	Planeado para a fase final

Quadro 8 — Estratégia de testes e níveis de evidência. Fonte: elaboração própria.

3.14.2 Testes com utilizadores reais

Está planeada uma validação estruturada com pessoas que não conheçam profundamente o projeto. O objetivo é medir se a aplicação é compreensível, intuitiva e adequada ao público-alvo, e não apenas confirmar execução sem erros. O guião prevê tarefas como descobrir se o computador está bem, localizar Análise de Segurança, iniciar uma Análise Rápida, encontrar o processo que mais utiliza memória, interpretar um resultado de scan, identificar um problema de proteção e pedir uma explicação à Matrix.

Para cada tarefa deverão ser registados sucesso, tempo aproximado, pedidos de ajuda, termos não compreendidos e observações. No final será recolhido feedback sobre nomes, navegação, estados de segurança e aparência. Até estes testes serem efetivamente realizados, opiniões informais não são tratadas como prova suficiente de usabilidade.

3.14.3 Estado atual das funcionalidades

Funcionalidade	Estado	Descrição
Monorepo e arquitetura principal	Implementado	Packages especializados e contratos partilhados.
Interface React e Electron	Implementado	Renderer, preload e processo principal.
Serviço local e named pipe	Implementado	Pedidos e eventos entre desktop e serviço.
ProtectionHealth	Implementado	Disponibilidade, degradação, erro e detalhe.
YARA-X	Implementado / cobertura limitada	Motor funcional; corpus validado ainda reduzido.
Base de hashes	Implementado / em evolução	SHA-256 e assinaturas locais; proveniência em limpeza.
ClamAV	Parcial / degradado	Binário presente; base de assinaturas não operacional.
Windows Firewall	Implementado	Consulta separada do filtro WFP próprio.
Filtro WFP CyberSafe	Indisponível / experimental	Não equivale a Firewall do Windows desligado.
PcState e findings	Implementado	Categorias, prioridades e deduplicação.
Dashboard	Implementado	Estado geral, proteção, findings e métricas.
Telemetria em memória	Implementado	Ring buffer e sparklines; sem persistência.
PC Center	Implementado / em validação	Dados reais; sensores ainda parciais.
Temperaturas	Parcial	CPU indisponível no equipamento de desenvolvimento.
Scanner no serviço	Implementado	Rápida, Profunda, ficheiro, pasta e ZIP com eventos.
Análise no Electron	Implementado / em validação	Ficheiro único validado; Profunda e ZIP aninhado em QA.
Quarentena	Implementado	Mover, restaurar, eliminar e metadados.
Matrix	Implementado / parcial	Políticas existentes; contexto global em evolução.
Onboarding e temas	Implementado	Persistência local e várias identidades visuais.
Autenticação básica	Implementado	Registo, login, JWT, SQLite e role formal.
Extensão de navegador	Parcial	Manifest V3 e lógica inicial; integração por validar.

Notificações	Parcial	Infraestrutura inicial e alguns eventos.
Autostart	Validação pendente	Implementado; falta validar build empacotada.
Relatório interno	Parcial	Layout existente; fontes centrais ainda não uniformes.
Native Engine	Ativo em DEV / packaging pendente	C++ integrado; build antiga empacotada está desatualizada.
ETW e AMSI	Experimental	Investigação/protótipos, sem proteção completa.
CyberSafe Business	Fase 1 implementada	Business Security Center e endpoint; multi-endpoint futuro.
Gestor de Tarefas	Implementado / em validação	Término validado; latência da lista instável.

Tabela 1 — Estado atual das funcionalidades do CyberSafe Assistant. Estado documental revisto em 04/10/2026, com base no último checkpoint técnico comprovado.

3.14.4 Evidências técnicas principais

Validação	Resultado	Evidência/observação
WS-B — estado dos motores e copy	PASS humano	YARA 1 regra; Hash DB 4 assinaturas; ClamAV degradado; Native ativo; Firewall/WFP separados.
Gestor de Tarefas — terminar Notepad	PASS humano	Processo encerrado por PID, feedback e log com estado ok.
Native SHA-256	PASS	Digest corrigido e alinhado com ferramentas independentes.
Auditoria de contaminação do hash	PASS	Stores auditadas sem ocorrência do digest incorreto em decisões persistidas.
Qualidade de deteção Bun	PASS regressão	Bun deixou de ser crítico; EICAR manteve comportamento crítico em teste controlado.
Windows Firewall	PASS leitura direta	Perfis Domain, Private e Public ativos no ambiente.
WFP CyberSafe	ABERTO	Filtro próprio indisponível; UI explica que não significa Firewall do Windows desligado.
ClamAV freshclam	BLOCKER identificado	freshclam.conf não pôde ser aberto/interpretado; sem base operacional.
Scanner — gates finais	PENDENTE	Folder/ZIP pequenos, cancelamento, timers e estados completos a fechar em DEV.

Quadro 9 — Síntese de evidências técnicas e manuais do checkpoint.

3.14.5 Gate de coordenação de escrita na base de dados ClamAV

Foi introduzido um gate de coordenação de escrita na base de dados ClamAV (packages/service/src/clam-database-gate.ts). O mecanismo distingue três causas de adiamento: serviço-a-encerrar (ambiente já descartado), busy-scans (leases de scan ativas) e outro-escritor. Antes desta correção, um adiamento por serviço a encerrar era classificado incorretamente como busy-scans, afetando o diagnóstico interno de serviço. Foram adicionados seis testes unitários direcionados a esta lógica, integrados na suite p1-1-f17-refusal-cause. O wide gate completo resultou verde: core 494, service 351, app 485, native-windows 65, native-engine 25, desktop 29, api 29; typecheck raiz exit 0; stress de 240 ciclos, 718 análises, 480 escritores e zero violações de admissão. O comportamento de rótulo de adiamento não foi observado diretamente em runtime manual.

3.15 Dificuldades encontradas e soluções aplicadas

3.15.1 Ecrã preto no Electron

Um ecrã preto foi causado por mudança da ordem de React Hooks entre renders: um useMemo encontrava-se depois de um retorno antecipado associado ao arranque. A correção consistiu em mover todos os hooks para antes dos retornos condicionais, demonstrando como uma regra estrutural do React pode impedir toda a interface de renderizar.

3.15.2 CSS e especificidade dos temas

Foram encontrados erros de sintaxe CSS, incluindo uma chaveta em falta, e conflitos de especificidade que faziam o tema substituir a cor de destaque no modo claro. A solução combinou correção estrutural, seletores com especificidade equivalente e testes das combinações de modo, tema e accent.

3.15.3 Integração e progresso da Análise de Segurança

O serviço funcionava isoladamente, mas a interface perdia eventos. A correção passou por subscrição estável, buffering de eventos precoces e ressincronização a partir da sessão no serviço. Uma segunda inconsistência entre threat.detected e threatsFound foi corrigida. Também foi introduzido refreshBuiltinIfStale para atualizar uma assinatura built-in antiga na Hash DB quando o ficheiro persistido já existia.

3.15.4 Sensores de temperatura

WMI e ACPI não produziram leitura utilizável da CPU no hardware de desenvolvimento. A aplicação chegou a receber uma leitura temporária de GPU, depois indisponível. O projeto preferiu remover uma dependência de baixo nível insegura e apresentar "Indisponível" em vez de manter valores antigos ou estimados.

3.15.5 Junctions e erros EPERM

Junctions do Windows em pastas de compatibilidade produziram EPERM e risco de ciclos. A abordagem consiste em não seguir reparse points inadequados, tolerar erros transitórios esperados e

limitar logs repetidos, sem ignorar completamente a pasta Temp, que continua relevante para segurança.

3.15.6 Encoding e texto corrompido

Algumas edições PowerShell produziram texto corrompido por incompatibilidade de encoding. A disciplina passou a exigir UTF-8 consistente, verificação do diff e ferramentas que preservem explicitamente a codificação.

3.15.7 Recuperação e disciplina de Git

Um comando repôs FileScannerPage.tsx para o estado do repositório e eliminou alterações locais não commitadas. Parte do trabalho teve de ser reconstruída. O incidente reforçou commits menores, backups e revisão do estado do Git antes de operações destrutivas.

3.15.8 Memória e processos de desenvolvimento

Uma percentagem elevada de memória foi inicialmente associada ao CyberSafe, mas a métrica representava o total do computador. Foi criado scripts/memory-report.mjs para distinguir processos do produto, ferramentas de desenvolvimento e outras aplicações. Ainda não existe medição longa suficiente para afirmar ausência total de fugas de memória.

3.15.9 Gestor de Tarefas: lista e término de processos

Um processo recém-aberto podia demorar demasiado a aparecer devido a ciclos de polling e cache no renderer. Polling adaptativo e enumeração single-flight reduziram algumas medições para cerca de dois segundos, mas testes humanos posteriores voltaram a observar atrasos superiores a dez segundos. A latência permanece instável.

O término por PID foi validado: o backend usa taskkill /F /T /PID, protege processos críticos e distingue SELF, ANCESTOR e INTERNAL. Num teste com Notepad, o processo encerrou, a mensagem de sucesso foi apresentada e o log registou o PID correto. A conclusão deste fluxo não elimina a dívida separada da atualização da lista.

3.15.10 Correção do SHA-256 nativo e auditoria de contaminação

O Native Engine contava duas vezes o total de bytes na fase final de padding para ficheiros cujo tamanho era múltiplo de 64 bytes. O problema foi reproduzido com Bun. Após a correção, o digest coincidiu com ferramentas independentes. Foi auditada a pasta %USERPROFILE%\.cybersafe-service e não foi encontrada a versão incorreta do hash em decisões persistidas, porque a arquitetura principal gravava hashes calculados pela camada Node/core.

3.15.11 Falso positivo do Bun e qualidade de deteção

O executável legítimo Bun chegou a ser classificado como 100/critical e colocado em quarentena. A análise mostrou combinação excessiva de regra YARA genérica, assinatura indisponível e entropia. A correção eliminou duplicação de fatores, deixou de penalizar estados desconhecidos e exigiu maior

corroboração. Bun passou a baixo risco/seguro, enquanto o EICAR manteve comportamento crítico em testes controlados.

3.15.12 Contrato WFP, Windows Firewall e honestidade da interface

Uma auditoria encontrou ambiguidade entre a propriedade usada para WFP e a designação "firewall", além de uma condição lógica que podia suprimir um finding do Windows Firewall. O contrato passou a utilizar wfp e a interface separa claramente "Windows Firewall" de "Filtro WFP (CyberSafe)". Esta correção demonstra que testar semântica e copy é tão importante como testar o backend.

3.15.13 Coordenação freshclam e gate de escrita na base de dados ClamAV

Durante um reteste manual de scanner em lote, verificou-se que o adiamento de atualização da base de dados ClamAV podia ser classificado como busy-scans mesmo quando a causa real era o encerramento do serviço. Esta ambiguidade dificultava o diagnóstico de falhas de freshclam. A solução consistiu em separar explicitamente as três causas de adiamento no gate de coordenação e garantir que cada estado interno é identificado pela evidência correta. O resultado não altera o comportamento de bloqueio nem a interface visível ao utilizador, mas melhora a rastreabilidade de incidentes relacionados com a atualização de assinaturas.

3.16 Avaliação provisória dos resultados

O projeto apresenta um núcleo funcional significativamente mais maduro do que a proposta inicial. A arquitetura em monorepo, a interface React/Electron, o serviço local, o named pipe, PcState, ProtectionHealth, Dashboard, telemetria, quarentena, scanner, autenticação e primeira fase Business estão implementados em diferentes níveis e foram sujeitos a testes automatizados e validação de runtime.

A passagem de qualidade do final de setembro produziu resultados concretos: término de processos validado, SHA-256 nativo corrigido, ausência de contaminação persistida verificada, falso positivo do Bun corrigido sem quebrar deteção de referência e contrato WFP/Firewall corrigido. No Dashboard, os motores passaram a mostrar estado e cobertura separadamente.

A cobertura real continua desigual. Três motores ativos em quatro não significam três quartos da proteção de um produto comercial. O resultado mais importante é a capacidade de mostrar honestamente o que está disponível e o que falta.

O reteste de 2026-10-03 confirmou a estabilização dos recursos do motor ClamAV: a RAM manteve-se em torno de 60–62 % e o pico de processos clamscan observado no Gestor de Tarefas foi de um. O pico de esgotamento de RAM que bloqueava o sistema não se repetiu neste reteste. A validação é parcial: o mesmo reteste não completou o scanner em lote real, pelo que a estabilidade de recursos não pode ser declarada para análises de grande volume em condições reais.

3.17 Limitações atuais

• O ClamAV está instalado, mas não possui base de assinaturas operacional; freshclam.conf é o primeiro bloqueio confirmado.

• O YARA-X está funcional, mas a cobertura validada permanece muito pequena e precisa de expansão controlada, com origem e licença conhecidas.

• A Hash DB possui apenas quatro entradas locais e parte da proveniência/rotulagem precisa de revisão antes de ser tratada como base de produção.

• O filtro WFP próprio do CyberSafe permanece indisponível; isto não significa que o Windows Firewall esteja desligado.

• A latência de atualização do Gestor de Tarefas continua instável, apesar de o término por PID estar validado.

• Os gates finais do scanner em DEV — pasta pequena, ZIP pequeno, cancelamento, timer e estados completos — ainda precisam de uma passagem concentrada de QA.

• A leitura de temperaturas não é fiável em todos os equipamentos; a interface mantém "Indisponível" quando não existe leitura real.

• O binário do Native Engine numa pasta win-unpacked anterior à correção SHA-256 está desatualizado; o packaging permanece congelado até rebuild e smoke test.

• A extensão, notificações, sincronização de preferências e várias funções Business continuam parciais ou planeadas.

• A autenticação e a fase Business não constituem uma infraestrutura multi-endpoint pronta para produção.

• Os testes estruturados com utilizadores externos ainda não foram realizados.

• O projeto é uma PAP e não substitui antivírus comercial, EDR empresarial ou ferramenta forense.

• O pipeline de análise em lote real não completa: num reteste de 2026-10-03, tanto a análise personalizada de cinco ficheiros como a análise rápida de 800 ficheiros foram aceites pelo serviço mas ficaram paradas em 0/N sem progressão nem erro visível durante mais de 60 segundos (SCANNER_REAL_BATCH_STALL). A origem do bloqueio não está identificada e não pode ser atribuída a nenhum workstream anterior sem prova.

3.18 Registo de riscos técnicos

O registo seguinte transforma limitações e incidentes observados num conjunto de riscos acompanháveis. A classificação é qualitativa e serve para priorização interna da PAP, não para certificação formal.

Risco	Prob.	Impacto	Evidência/origem	Controlo principal	Estado
Cobertura de deteção insuficiente	Alta	Alta	YARA/Hash DB ainda pequenos; ClamAV sem base	Expandir corpus com proveniência/licença e ativar base ClamAV	Aberto
Falso positivo elevado	Média	Alta	Caso Bun demonstrou risco real	Corroboração de sinais, testes de regressão e explicação de incerteza	Mitigado / vigiar
Build empacotada com binário desatualizado	Média	Alta	Native Engine antigo em win-unpacked	Rebuild limpo, checksum e smoke test do pacote	Aberto
IPC ou preload excessivamente permissivo	Baixa/Média	Alta	Risco inerente a Electron; auditoria final ainda necessária	Context isolation, sandbox, sender validation, CSP e superfície mínima	A verificar
Logs com dados sensíveis	Baixa/Média	Média/Alta	Diagnóstico pode incluir paths/tokens	Sanitização, exclusão de segredos e revisão do pacote	Em controlo
Latência do Gestor de Tarefas	Alta	Média	Atrasos >10 s reapareceram em testes humanos	Medição, cache/polling revistos e refresh explícito	Aberto
Sensores incorretos ou stale	Média	Média	Leitura de GPU oscilou; CPU indisponível	TTL curto, identificação de device e “Indisponível” quando não fiável	Controlado por estado
Perda de trabalho por Git	Baixa	Média	Incidente com checkout/reset de ficheiro	Commits pequenos, backup e proteção contra comandos destrutivos	Mitigado
Resposta de IA não fundamentada	Média	Média/Alta	Matrix pode extrapolar sem contexto	Grounding, política de pedir contexto e separar observação/inferência	Em evolução
Cadeia de dependências	Média	Alta	Electron/npm/motores externos e C++	Inventário, lockfiles, atualização controlada, provenance e revisão de advisories	Continuado
Stall do scanner em lote real	Alta	Alta	Reteste 2026-10-03: análise aceite e parada	Investigação read-only planeada; não atribuir a	Aberto

			em 0/N, >60 s; origem não identificada	outro workstream sem prova	

Quadro 10 — Registo de riscos técnicos e controlos. Fonte: elaboração própria.

3.19 Trabalho futuro

3.19.1 Motores e cobertura de deteção

A prioridade imediata é tornar a cobertura auditável. No ClamAV, definir freshclam.conf, validar o caminho da base e executar atualização controlada. Na Hash DB, rever as quatro entradas quanto a proveniência, família e licença. No YARA-X, expandir apenas com regras autorais ou fontes de licença clara, acompanhadas de testes de falsos positivos.

3.19.2 Scanner e validação de runtime

Fechar pasta pequena, ZIP pequeno, cancelamento, estados, timer, origem da deteção e consistência entre eventos e resultado final. Scans profundos da máquina só devem ser usados se acrescentarem evidência necessária e o ambiente estiver estável.

3.19.3 Usabilidade e validação com utilizadores

Executar o guião já definido sem coaching, medir taxa de sucesso, tempo, dúvidas, termos incompreendidos e ações escolhidas. Melhorias devem ser ligadas diretamente às observações dos participantes.

3.19.4 Empacotamento e entrega

Retomar packaging depois dos gates principais: reconstruir Native Engine corrigido, confirmar ausência de logs privados, validar paths de resources, arranque do serviço, autostart, notificações, opções de segurança Electron e smoke test do win-unpacked. Instalador e assinatura digital pertencem à etapa de entrega.

3.19.5 Saúde do PC e sensores

Continuar a arquitetura de providers seguros. Saúde de bateria, ciclos e desgaste são candidatos futuros, mas não devem ultrapassar motores, scanner, usabilidade e build final na prioridade da PAP.

3.19.6 CyberSafe Business e rede

A vertente Business poderá evoluir para inventário autorizado, eventos de rede, deteção de anomalias, múltiplos endpoints, políticas e alertas centralizados. Qualquer evolução deve respeitar autorização, minimização de dados e limites legais.

3.19.7 Evolução da Matrix

A Matrix deverá consumir PcState, ProtectionHealth, findings e resultados de ações de forma mais direta, produzindo explicações curtas e fundamentadas. A integração deve preservar sanitização e

distinguir dados observados, inferências e informação indisponível.

4. AVALIAÇÃO E DISCUSSÃO

4.1 Cumprimento dos objetivos

A comparação entre objetivos e evidência mostra que a arquitetura principal, o estado dos mecanismos de proteção, o scanner de base, o Dashboard, o PC Center, PcState, quarentena, autenticação e assistência já ultrapassam a fase meramente conceptual. Ao mesmo tempo, alguns objetivos foram parcialmente cumpridos porque a validação final depende de motores externos, sensores, packaging e testes com utilizadores.

Objetivo	Evidência atual	Avaliação
Interface acessível para utilizadores comuns	Dashboard, copy humanizada, estados traduzidos, onboarding	Cumprido em implementação; falta teste de usabilidade externo
Análise de ficheiros e motores	Scanner, YARA-X, Hash DB, Native Engine, ClamAV degradado	Parcialmente cumprido devido à cobertura
Distinguir "sem deteções" de "seguro"	ProtectionHealth, cobertura explícita e textos de interface	Cumprido como princípio e design
Monitorizar hardware/processos/rede	PC Center, telemetria e integrações Windows	Cumprido parcialmente; temperaturas limitadas
Findings e prioridade	PcState com categorias e severidades	Cumprido
Quarentena	Mover, restaurar, eliminar e metadados	Cumprido no âmbito implementado
Assistência contextual	Matrix e políticas de grounding	Parcial; integração global em evolução
Arquitetura modular	Monorepo, packages, IPC, service/core	Cumprido
Testes e transparência	Suites, probes, testes manuais, riscos e limitações	Cumprido como processo; validação final ainda por executar
Módulos complementares	Extensão, Business, notificações, autenticação	Parcial / fase inicial

Quadro 11 — Objetivos do projeto e evidência disponível. Fonte: elaboração própria.

4.2 Cobertura, confiança e honestidade técnica

A principal aprendizagem técnica da fase recente é que "motor ativo" e "proteção suficiente" são conceitos diferentes. O caso mais evidente é YARA-X: o mecanismo funciona, mas a cobertura validada é pequena. O mesmo se aplica à Hash DB. ClamAV demonstra o inverso: o binário existe, mas sem base de assinaturas não deve ser contado como proteção plena. Esta separação torna o produto menos impressionante num número isolado, mas mais correto.

O Protection Score é útil como síntese interna apenas se for acompanhado por detalhe. A discussão não deve centrar-se em "80/100", mas em quais motores produziram esse valor, qual a sua cobertura, qual está degradado e que ações são necessárias. Esta é uma área em que a versão atual do projeto evoluiu de uma estética de antivírus para uma lógica de confiança auditável.

4.3 Usabilidade e comunicação de risco

A estrutura dado → significado → prioridade → orientação aproxima o produto da literatura de usable security, porque tenta reduzir jargão sem eliminar a evidência. A correção WFP/Firewall é um exemplo significativo: tecnicamente, ambos pertencem ao domínio de rede, mas para o utilizador a frase "firewall indisponível" podia induzir uma decisão errada. Separar os conceitos é uma melhoria de segurança, não apenas de copy.

A avaliação ainda tem uma lacuna: os testes foram realizados sobretudo por quem conhece o sistema. A próxima etapa precisa de utilizadores externos, precisamente porque um autor tende a conhecer a localização das funções, os termos e o significado dos estados. A usabilidade só pode ser defendida com maior confiança depois dessa validação.

4.4 Segurança do próprio software e sustentabilidade

Uma aplicação de segurança aumenta a sua própria superfície de ataque porque precisa de observar ficheiros, processos, rede e componentes do sistema. Por isso, a robustez do produto não depende apenas da capacidade de detetar malware. Também depende da segurança do Electron, da integridade do named pipe, da atualização de dependências, da proveniência das regras, da qualidade dos logs, do packaging e da resposta a vulnerabilidades.

O SSDF e o CRA ajudam a enquadrar este ponto. Embora a PAP não declare conformidade, a evolução futura beneficia de práticas que já fazem sentido tecnicamente: inventário de componentes, atualização segura, configuração conservadora por defeito, documentação de vulnerabilidades, testes regulares e suporte a correções. O facto de um helper de temperatura inseguro ter sido removido e de uma build nativa desatualizada ter bloqueado o packaging são exemplos concretos de decisões coerentes com essa filosofia.

4.5 Aprendizagens e valor da PAP

A PAP permitiu trabalhar frontend, backend, arquitetura, integração Windows, C++, comunicação entre processos, testes, Git, cibersegurança, experiência do utilizador e utilização responsável de IA num único projeto. Muitos dos resultados mais importantes surgiram de falhas: o ecrã preto ensinou

disciplina de Hooks; a perda de eventos mostrou a importância da ordem temporal; o SHA-256 e o Bun evidenciaram necessidade de testes de referência; WFP/Firewall mostrou que uma nomenclatura errada pode tornar uma interface tecnicamente enganadora.

O valor do projeto não está em afirmar que foi construído um antivírus equivalente a produtos comerciais. Está na demonstração de integração, na evolução de arquitetura, na capacidade de medir o que funciona, na documentação do que ainda falta e na construção de uma interface que procura ajudar o utilizador sem lhe prometer mais do que a evidência permite.

5. CONCLUSÃO

O desenvolvimento do CyberSafe Assistant permitiu transformar uma ideia inicial de antivírus num projeto mais amplo, centrado na relação entre segurança, saúde do sistema e compreensão do utilizador. A maior evolução não ocorreu apenas na quantidade de funcionalidades, mas na forma como o produto passou a distinguir deteção, cobertura, disponibilidade, confiança e orientação.

A arquitetura modular em React, Electron, TypeScript, Node.js, named pipe e componentes nativos permitiu integrar diferentes áreas sem colocar todas as responsabilidades na interface. ProtectionHealth e PcState tornaram-se fontes importantes para reduzir contradições. A experiência com YARA-X, Hash DB, ClamAV, Native Engine, Windows Firewall e WFP demonstrou que cada componente precisa de ser medido e explicado separadamente.

As dificuldades encontradas contribuíram diretamente para a aprendizagem. React Hooks, CSS, encoding, perda de eventos do scanner, sensores, SHA-256, falso positivo do Bun, latência do Gestor de Tarefas e ambiguidade WFP/Firewall mostraram que um problema raramente se limita ao ponto onde o sintoma aparece. Foi necessário formular hipóteses, isolar camadas, criar testes, reproduzir problemas e validar novamente em runtime.

Um dos resultados mais relevantes foi a adoção de uma política de honestidade técnica. O CyberSafe passou a mostrar que um motor pode estar ativo com cobertura limitada, que "sem deteções" não equivale a "computador seguro", que um sensor desconhecido deve permanecer indisponível e que um filtro próprio do CyberSafe não é o Windows Firewall. O enquadramento revisto reforça esta direção com referências de gestão de risco, usable security, desenvolvimento seguro, privacidade e regulamentação europeia.

O projeto encontra-se avançado, mas ainda não final. ClamAV, corpus YARA, proveniência da Hash DB, gates finais do scanner, latência do Gestor de Tarefas, sensores, auditoria de segurança Electron, testes de usabilidade e empacotamento continuam a exigir trabalho. Assumir essas limitações é parte do resultado da PAP: demonstrar o que falta, explicar por que falta e definir um caminho de validação constitui desenvolvimento responsável.

O CyberSafe Assistant permanece em evolução, mas já demonstra de forma concreta a ideia que orienta o produto: o computador produz dados técnicos; o CyberSafe procura transformá-los em significado, prioridade e orientação, sem ocultar o grau de confiança disponível.

6. REFERÊNCIAS BIBLIOGRÁFICAS

Autio, C., Schwartz, R., Dunietz, J., Jain, S., Stanley, M., Tabassi, E., Hall, P., & Roberts, K. (2024). Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile (NIST AI 600-1). National Institute of Standards and Technology.https://doi.org/10.6028/NIST.AI.600-1

ClamAV. (s.d.). Configuration: freshclam.conf. ClamAV Documentation. Acedido em 4 de outubro de 2026.https://docs.clamav.net/manual/Usage/Configuration.html

Cranor, L. F. (2008). A framework for reasoning about the human in the loop. Usability, Psychology, and Security 2008 (UPSEC 08), USENIX Association.https://www.usenix.org/conference/upsec-08/framework-reasoning-about-human-loop

Di Nocera, F. (2026). Usable security: a (re)definition and a research agenda. Theoretical Issues in Ergonomics Science, 27(2), 118–131.https://doi.org/10.1080/1463922X.2025.2597035

Di Nocera, F., Tempestini, G., & Orsini, M. (2023). Usable Security: A Systematic Literature Review. Information, 14(12), 641.https://doi.org/10.3390/info14120641

Electron. (s.d.). Security: security recommendations. Electron Documentation. Acedido em 4 de outubro de 2026.https://www.electronjs.org/docs/latest/tutorial/security

European Parliament and Council of the European Union. (2016). Regulation (EU) 2016/679 (General Data Protection Regulation), Articles 5 and 25.https://eur-lex.europa.eu/eli/reg/2016/679/pt

European Parliament and Council of the European Union. (2024). Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act).https://eur-lex.europa.eu/eli/reg/2024/2847/oj/por

Furnell, S. (2024). Usable Cybersecurity: a Contradiction in Terms? Interacting with Computers, 36(1), 3–15.https://doi.org/10.1093/iwc/iwad035

Gibert, D., Mateu, C., & Planes, J. (2020). The rise of machine learning for detection and classification of malware: Research developments, trends and challenges. Journal of Network and Computer Applications, 153, 102526.https://doi.org/10.1016/j.jnca.2019.102526

Google Chrome Developers. (s.d.). Declare permissions — Chrome Extensions / Manifest V3. Acedido em 4 de outubro de 2026.https://developer.chrome.com/docs/extensions/develop/concepts/declare-permissions

Microsoft. (s.d.-a). Windows Filtering Platform. Microsoft Learn. Acedido em 4 de outubro de 2026.https://learn.microsoft.com/windows/win32/fwp/windows-filtering-platform-start-page

Microsoft. (s.d.-b). Windows Firewall tools. Microsoft Learn. Acedido em 4 de outubro de 2026.https://learn.microsoft.com/windows/security/operating-system-security/network-security/windows-firewall/tools

National Institute of Standards and Technology. (2024). The NIST Cybersecurity Framework (CSF) 2.0 (NIST CSWP 29).https://doi.org/10.6028/NIST.CSWP.29

Nelson, A., Rekhi, S., Souppaya, M., & Scarfone, K. (2025). Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile (NIST SP 800-61 Rev. 3). National Institute of Standards and Technology.https://doi.org/10.6028/NIST.SP.800-61r3

OWASP Foundation. (s.d.). Logging Cheat Sheet. OWASP Cheat Sheet Series. Acedido em 4 de outubro de 2026.https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

Scarfone, K., Souppaya, M., & Dodson, D. (2022). Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities (NIST SP 800-218).https://doi.org/10.6028/NIST.SP.800-218

Souppaya, M., & Scarfone, K. (2013). Guide to Malware Incident Prevention and Handling for Desktops and Laptops (NIST SP 800-83 Rev. 1). National Institute of Standards and Technology.https://doi.org/10.6028/NIST.SP.800-83r1

Souri, A., & Hosseini, R. (2018). A state-of-the-art survey of malware detection approaches using data mining techniques. Human-centric Computing and Information Sciences, 8, 3.https://doi.org/10.1186/s13673-018-0125-x

Tabassi, E. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0) (NIST AI 100-1). National Institute of Standards and Technology.https://doi.org/10.6028/NIST.AI.100-1

Theofanos, M. (2020). Is Usable Security an Oxymoron? Computer, 53(2), 71–74.https://doi.org/10.1109/MC.2019.2954075

VirusTotal. (s.d.). YARA-X Documentation. Acedido em 4 de outubro de 2026.https://virustotal.github.io/yara-x/docs/

ANEXOS

Os anexos reúnem materiais complementares que apoiam a compreensão e a validação do projeto sem interromper o desenvolvimento do texto principal. As capturas correspondem ao material disponibilizado na versão-base do relatório e devem ser substituídas por evidência mais recente se novos testes forem realizados antes da entrega final.

Anexo A — Arquitetura e pontos de validação

Camada	Responsabilidade	Comunicação/contrato	Falhas observadas e controlos
React / App	Apresentação, interação e estado visual	APIs expostas pelo preload	Copy, timers e re-render; testes de componentes e runtime.
Electron / Preload	Fronteira desktop, IPC e lifecycle	IPC tipado / eventos	Listeners tardios, paths de packaging; validar sender e ressincronizar.
Service	Orquestra scans, processos, proteção e eventos	Named pipe / RPC	Timeouts, erros Windows, scheduler; estados estruturados e logs.
Core	Análise, scoring, PcState e ProtectionHealth	Tipos partilhados	Falsos positivos, regras e deduplicação; testes e canários.
Native / Windows	Hash, PE, processos, firewall e sistema	Process spawn / PowerShell / APIs	Encoding, permissões, paths e SHA; probes independentes.
Motores externos	ClamAV, YARA-X e feeds	Ficheiros / CLI / HTTP	Configuração, licença, cobertura e autenticação; estado degradado quando falta evidência.

Anexo A — Mapa de camadas, ligações e pontos de validação. Fonte: elaboração própria.

Anexo B — Capturas de ecrã

As capturas seguintes documentam estados reais observados durante o desenvolvimento. Devem ser interpretadas juntamente com as limitações descritas no texto: um resultado sem deteções não é garantia absoluta e o estado de um motor deve ser lido em conjunto com a cobertura disponível.

Figura C1 — Resultado da Análise Rápida do Computador: 800 ficheiros verificados, 747 sem deteções, 53 ignorados por exclusões e 0 ameaças encontradas.

Figura C2 — Estado da Proteção após a passagem WS-B: motores, Windows Firewall e WFP apresentados com estados separados, cobertura explícita e contexto humano.

Figura C3 — Validação end-to-end com canário benigno: um ficheiro verificado, uma ameaça encontrada e alerta apresentado de forma coerente.

Anexo C — Excertos técnicos

C.1 Estados do scanner

idle → preparing → scanning → completed | cancelled | failed. O serviço mantém a sessão como fonte de verdade e a interface pode recuperar o estado depois de sair e regressar à página.

C.2 Guarda de término de processos

Antes de terminar um PID, o serviço verifica processos protegidos e relações com o próprio CyberSafe. SELF significa o próprio processo; ANCESTOR, um ancestral do CyberSafe; INTERNAL, um descendente do CyberSafe. Partilhar apenas um ancestral comum não é suficiente para classificar outro processo como interno.

C.3 Saúde dos motores

Cada componente devolve estado + detalhe. A camada de apresentação converte o detalhe técnico em contexto humano e mantém paths, mensagens CLI e enums crus nos detalhes técnicos. Cobertura — por exemplo, número de regras ou assinaturas — é apresentada separadamente do estado Ativo.

C.4 Princípio de score

O score de proteção é limitado pela disponibilidade dos motores e pelas ameaças observadas. O objetivo não é provar segurança, mas comunicar confiança relativa dentro da cobertura existente. PcState e ProtectionHealth permanecem conceitos distintos.

Anexo D — Planeamento e evolução

Período	Marco	Efeito no projeto
11.º ano	Ideia inicial de antivírus	Definição do problema e primeiros módulos de análise.
2025–2026	Migração para arquitetura modular / monorepo	Separação entre app, Electron, serviço, core, API e integrações.
1.º semestre de 2026	PC Center, ProtectionHealth, PcState, Matrix e scanner	Projeto deixa de ser apenas antivírus e passa a integrar saúde do PC.
Setembro de 2026	Passagens intensivas de QA	Scanner, Task Manager, deteção, packaging, WFP/Firewall e estados reais dos motores.
30/09/2026	WS-B PASS humano	Copy dos motores humanizada e cobertura limitada apresentada explicitamente.
30/09/2026	Checkpoint organizador	206 tarefas: 102 concluídas, 27 em curso, 77 planeadas.
01/10/2026	Atualização do relatório v8	Fundamentação, arquitetura, resultados e limitações atualizados.
04/10/2026	Revisão documental v9	Expansão de literatura, sobreposições, privacidade, SSDF, CRA, riscos e discussão.

Anexo D — Cronologia resumida da evolução do CyberSafe Assistant.

Anexo E — Checklist antes da entrega final

- Reexecutar suites e registar data, comandos, ambiente e resultados finais.
- Fechar gates pequenos do scanner em DEV e guardar evidência.
- Corrigir/configurar freshclam e confirmar base de assinaturas ClamAV, ou manter estado degradado documentado.
- Rever proveniência/licença da Hash DB e do corpus YARA-X.
- Rebuild controlado do Native Engine e smoke test do pacote win-unpacked.
- Auditar configurações Electron: context isolation, sandbox, CSP, sender validation e navegação.
- Executar testes com utilizadores externos e acrescentar resultados ao relatório.
- Rever logs, paths e ficheiros do pacote para impedir exposição desnecessária de dados.
- Sincronizar Git/GitHub com um checkpoint final identificável e manter tag/commit de entrega.
- Atualizar screenshots e números de testes apenas com evidência real do checkpoint final.